

Yesterday 3:34 PM

Reverse a String.
Reverse words in a String.
Check palindrome.
Find duplicate characters.
Find first non-repeating character.
Check anagrams.
Count character frequency.
Find missing number.
Find duplicate number.
Move zeros to end.
Find second-largest number.

[Show more](#) 

Absolutely. For a **5-year Java interview**, don't just memorize the code—be ready to explain **why this approach, time complexity, space complexity, and edge cases**.

Below are optimized, interview-friendly Java solutions. I'll keep the code **simple enough to explain on a whiteboard**.

1. Reverse a String

Best approach: `StringBuilder`

 **Java**



```
public static String reverse(String str) {  
    return new StringBuilder(str).reverse().toString();  
}
```

Without built-in method

 **Java**



```
public static String reverse(String str) {  
    char[] chars = str.toCharArray();  
  
    int left = 0, right = chars.length - 1;  
  
    while (left < right) {  
        char temp = chars[left];  
        chars[left] = chars[right];  
        chars[right] = temp;  
  
        left++;  
        right--;  
    }  
  
    return new String(chars);  
}
```

Complexity:Time: $O(n)$ Space: $O(n)$ **Interview:** If asked to reverse without creating another string, use the `char[]` approach.

2. Reverse Words in a String

Input:

`"Java is powerful"`

Output:

`"powerful is Java"``</> Java`

```
public static String reverseWords(String str) {  
    String[] words = str.trim().split("\\s+");  
  
    StringBuilder result = new StringBuilder();  
  
    for (int i = words.length - 1; i >= 0; i--) {  
        result.append(words[i]);  
    }  
}
```

```
        if (i != 0) {  
            result.append(" ");  
        }  
    }  
  
    return result.toString();  
}
```

Complexity: $O(n)$ time, $O(n)$ space.

3. Check Palindrome

Example:

madam → true
hello → false



</> Java



```
public static boolean isPalindrome(String str) {  
    int left = 0;  
    int right = str.length() - 1;  
  
    while (left < right) {  
        if (str.charAt(left) != str.charAt(right)) {  
            return false;  
        }  
  
        left++;  
        right--;  
    }  
  
    return true;  
}
```

Complexity:

Time: $O(n)$

Space: $O(1)$

Interview point

Two-pointer technique is better than creating a reversed string because it uses constant extra space.

4. Find Duplicate Characters

</> Java



```
public static void findDuplicates(String str) {  
  
    Map<Character, Integer> map = new HashMap<>();  
  
    for (char ch : str.toCharArray()) {  
        map.put(ch, map.getOrDefault(ch, 0) + 1);  
    }  
  
    for (Map.Entry<Character, Integer> entry : map.entrySet()) {  
        if (entry.getValue() > 1) {  
            System.out.println(entry.getKey());  
        }  
    }  
}
```

Example:

programming



r
g
m

Complexity: $O(n)$ time, $O(k)$ space.

5. First Non-Repeating Character

Example:

swiss → w



Optimized/simple approach

</> Java



```
public static Character firstNonRepeating(String str) {
```

```
Map<Character, Integer> frequency = new LinkedHashMap<>();

for (char ch : str.toCharArray()) {
    frequency.put(ch, frequency.getOrDefault(ch, 0) + 1);
}

for (Map.Entry<Character, Integer> entry : frequency.entrySet()) {
    if (entry.getValue() == 1) {
        return entry.getKey();
    }
}

return null;
}
```

Why LinkedHashMap ?

Because it maintains **insertion order**.

Complexity: $O(n)$ time, $O(k)$ space.

6. Check Anagrams

Example:

listen
silent



Output:

true



Using frequency array

For lowercase English letters:

```
</> Java

public static boolean isAnagram(String s1, String s2) {

    if (s1.length() != s2.length()) {
        return false;
    }
}
```



```
int[] count = new int[26];

for (int i = 0; i < s1.length(); i++) {
    count[s1.charAt(i) - 'a']++;
    count[s2.charAt(i) - 'a']--;
}

for (int value : count) {
    if (value != 0) {
        return false;
    }
}

return true;
}
```

Time: $O(n)$

Space: $O(1)$

Interview point

For arbitrary Unicode characters, use a `HashMap<Character, Integer>` instead.

7. Count Character Frequency

 Java



```
public static Map<Character, Integer> frequency(String str) {

    Map<Character, Integer> map = new HashMap<>();

    for (char ch : str.toCharArray()) {
        map.put(ch, map.getOrDefault(ch, 0) + 1);
    }

    return map;
}
```

Example:

java



j=1

a=2
v=1

Time: $O(n)$

Space: $O(k)$

8. Find Missing Number

Given:

[1, 2, 3, 5]



Missing:

4



XOR solution

</> Java



```
public static int missingNumber(int[] nums) {  
  
    int n = nums.length;  
    int result = n;  
  
    for (int i = 0; i < n; i++) {  
        result ^= i;  
        result ^= nums[i];  
    }  
  
    return result;  
}
```

If numbers are from 0 to n .

Time: $O(n)$

Space: $O(1)$

Interview point

XOR avoids integer overflow that can occur with:

</> Java


$$n * (n + 1) / 2$$

for sufficiently large values.

9. Find Duplicate Number

Example:

`[1,3,4,2,2]`

Answer:

`2`

Floyd's Cycle Detection

This is the impressive interview solution.

</> Java



```
public static int findDuplicate(int[] nums) {  
  
    int slow = nums[0];  
    int fast = nums[0];  
  
    do {  
        slow = nums[slow];  
        fast = nums[nums[fast]];  
    } while (slow != fast);  
  
    slow = nums[0];  
  
    while (slow != fast) {  
        slow = nums[slow];  
        fast = nums[fast];  
    }  
  
    return slow;  
}
```

Time: $O(n)$

Space: $O(1)$

Why?

Treat array indexes and values like a linked list. The duplicate creates a cycle.

10. Move Zeros to End

Input:

[0,1,0,3,12]



Output:

[1,3,12,0,0]



 **Java**



```
public static void moveZeros(int[] nums) {  
  
    int index = 0;  
  
    for (int num : nums) {  
        if (num != 0) {  
            nums[index++] = num;  
        }  
    }  
  
    while (index < nums.length) {  
        nums[index++] = 0;  
    }  
}
```

Time: $O(n)$

Space: $O(1)$

Maintains the relative order of non-zero elements.

11. Find Second Largest Number

</> Java



```
public static int secondLargest(int[] nums) {

    int largest = Integer.MIN_VALUE;
    int second = Integer.MIN_VALUE;

    for (int num : nums) {

        if (num > largest) {
            second = largest;
            largest = num;
        } else if (num > second && num != largest) {
            second = num;
        }
    }

    if (second == Integer.MIN_VALUE) {
        throw new IllegalArgumentException("No second largest element");
    }

    return second;
}
```

Time: $O(n)$ **Space:** $O(1)$

Interview point

Don't sort the array.

Sorting gives:

 $O(n \log n)$ 

One pass gives:

 $O(n)$ 

12. Merge Two Sorted Arrays

Input:

[1,3,5]

[2,4,6]



Output:

[1,2,3,4,5,6]



 Java



```
public static int[] merge(int[] a, int[] b) {  
  
    int[] result = new int[a.length + b.length];  
  
    int i = 0, j = 0, k = 0;  
  
    while (i < a.length && j < b.length) {  
  
        if (a[i] <= b[j]) {  
            result[k++] = a[i++];  
        } else {  
            result[k++] = b[j++];  
        }  
    }  
  
    while (i < a.length) {  
        result[k++] = a[i++];  
    }  
  
    while (j < b.length) {  
        result[k++] = b[j++];  
    }  
  
    return result;  
}
```

Time: $O(n + m)$

Space: $O(n + m)$

13. Intersection of Arrays

Example:

[1,2,2,3]
[2,2,4]



If intersection should contain unique values:

[2]



</> **Java**



```
public static int[] intersection(int[] a, int[] b) {  
  
    Set<Integer> set = new HashSet<>();  
  
    for (int num : a) {  
        set.add(num);  
    }  
  
    Set<Integer> result = new HashSet<>();  
  
    for (int num : b) {  
        if (set.contains(num)) {  
            result.add(num);  
        }  
    }  
  
    return result.stream()  
        .mapToInt(Integer::intValue)  
        .toArray();  
}
```

Average time: $O(n + m)$

Space: $O(n)$

14. Rotate an Array

Rotate right by k .

Example:

[1,2,3,4,5,6,7]
 $k = 3$



Output:

[5,6,7,1,2,3,4]



Optimal reversal technique

</> Java



```
public static void rotate(int[] nums, int k) {

    k %= nums.length;

    reverse(nums, 0, nums.length - 1);
    reverse(nums, 0, k - 1);
    reverse(nums, k, nums.length - 1);
}

private static void reverse(int[] nums, int left, int right) {

    while (left < right) {
        int temp = nums[left];
        nums[left] = nums[right];
        nums[right] = temp;

        left++;
        right--;
    }
}
```

Time: $O(n)$

Space: $O(1)$

15. Maximum Subarray Sum

This is Kadane's Algorithm.

Input:

[-2,1,-3,4,-1,2,1,-5,4]



Answer:

6



Because:

$$4 + (-1) + 2 + 1 = 6$$


`</>` **Java**



```
public static int maxSubArray(int[] nums) {  
  
    int current = nums[0];  
    int max = nums[0];  
  
    for (int i = 1; i < nums.length; i++) {  
  
        current = Math.max(nums[i], current + nums[i]);  
  
        max = Math.max(max, current);  
    }  
  
    return max;  
}
```

Time: $O(n)$

Space: $O(1)$

Key idea

At every element:

Should I start a new subarray here or continue the existing one?

16. Longest Substring Without Repeating Characters

Example:

abcabcbb



Answer:

3



Substring:

abc



Sliding Window

</> Java



```
public static int longestUniqueSubstring(String s) {  
  
    Set<Character> set = new HashSet<>();  
  
    int left = 0;  
    int max = 0;  
  
    for (int right = 0; right < s.length(); right++) {  
  
        while (set.contains(s.charAt(right))) {  
            set.remove(s.charAt(left));  
            left++;  
        }  
  
        set.add(s.charAt(right));  
  
        max = Math.max(max, right - left + 1);  
    }  
  
    return max;  
}
```

Time: $O(n)$

Space: $O(k)$

Interview keywords

Remember:

Sliding Window + HashSet

17. Product of Array Except Self

Input:

[1,2,3,4]



Output:

[24,12,8,6]



Optimal $O(n)$

</> Java



```
public static int[] productExceptSelf(int[] nums) {

    int n = nums.length;
    int[] result = new int[n];

    result[0] = 1;

    // Prefix product
    for (int i = 1; i < n; i++) {
        result[i] = result[i - 1] * nums[i - 1];
    }

    int suffix = 1;

    // Suffix product
    for (int i = n - 1; i >= 0; i--) {

        result[i] *= suffix;

        suffix *= nums[i];
    }

    return result;
}
```

Time: $O(n)$

Extra space: $O(1)$ excluding output array.

Important

Don't use division.

Division fails when the array contains zero.

18. Three Sum

Find triplets whose sum is zero.

Example:

[-1,0,1,2,-1,-4]



Output:

[-1,-1,2]

[-1,0,1]



Optimized approach

</> Java



```
public static List<List<Integer>> threeSum(int[] nums) {

    Arrays.sort(nums);

    List<List<Integer>> result = new ArrayList<>();

    for (int i = 0; i < nums.length - 2; i++) {

        if (i > 0 && nums[i] == nums[i - 1]) {
            continue;
        }

        int left = i + 1;
        int right = nums.length - 1;

        while (left < right) {

            int sum = nums[i] + nums[left] + nums[right];

            if (sum == 0) {

                result.add(Arrays.asList(
                    nums[i],
                    nums[left],
                    nums[right]
                ));

                while (left < right && nums[left] == nums[left + 1]) {
                    left++;
                }
            }

            if (sum < 0) {
                left++;
            } else {
                right--;
            }
        }
    }

    return result;
}
```

```
        }

        while (left < right && nums[right] == nums[right - 1]) {
            right--;
        }

        left++;
        right--;

    } else if (sum < 0) {
        left++;
    } else {
        right--;
    }
}

return result;
}
```

Time: $O(n^2)$

Space: $O(1)$ excluding output.

Interview keywords

Sort + Two Pointer



19. Merge Intervals

Input:

```
[1,3]
[2,6]
[8,10]
[15,18]
```



Output:

```
[1,6]
[8,10]
[15,18]
```



</> Java



```
public static int[][] mergeIntervals(int[][] intervals) {

    if (intervals.length <= 1) {
        return intervals;
    }

    Arrays.sort(intervals, Comparator.comparingInt(a -> a[0]));

    List<int[]> result = new ArrayList<>();

    int start = intervals[0][0];
    int end = intervals[0][1];

    for (int i = 1; i < intervals.length; i++) {

        if (intervals[i][0] <= end) {
            end = Math.max(end, intervals[i][1]);
        } else {
            result.add(new int[]{start, end});

            start = intervals[i][0];
            end = intervals[i][1];
        }
    }

    result.add(new int[]{start, end});

    return result.toArray(new int[0][]);
}
```

Time: $O(n \log n)$ because of sorting.

Interview keywords:

Sort by start time → merge overlapping intervals.

20. Valid Parentheses

Input:

"{[()]}"



Output:

true



</> Java



```
public static boolean isValid(String s) {  
  
    Deque<Character> stack = new ArrayDeque<>();  
  
    for (char ch : s.toCharArray()) {  
  
        if (ch == '(') {  
            stack.push(')');  
        } else if (ch == '[') {  
            stack.push(']');  
        } else if (ch == '{') {  
            stack.push('}');  
        } else {  
  
            if (stack.isEmpty() || stack.pop() != ch) {  
                return false;  
            }  
        }  
    }  
  
    return stack.isEmpty();  
}
```

Time: $O(n)$

Space: $O(n)$

Nice interview trick

Instead of pushing opening brackets, push the **expected closing bracket**.

21. Implement LRU Cache

This is an important 5-year Java interview question.

Java already provides:

</> Java



`LinkedHashMap`

which can implement LRU easily.

`</>` Java



```
class LRUCache<K, V> extends LinkedHashMap<K, V> {  
  
    private final int capacity;  
  
    public LRUCache(int capacity) {  
        super(capacity, 0.75f, true);  
        this.capacity = capacity;  
    }  
  
    @Override  
    protected boolean removeEldestEntry(  
        Map.Entry<K, V> eldest) {  
  
        return size() > capacity;  
    }  
}
```

Usage:

`</>` Java



```
LRUCache<Integer, String> cache = new LRUCache<>(3);  
  
cache.put(1, "A");  
cache.put(2, "B");  
cache.put(3, "C");  
  
cache.get(1);  
  
cache.put(4, "D");
```

Entry 2 becomes the least recently used and can be removed.

Complexity

`get()` → $O(1)$ average

`put()` → $O(1)$ average

Interview follow-up

How would you implement it manually?

Answer:

HashMap + Doubly Linked List



HashMap gives $O(1)$ lookup.

Doubly linked list gives $O(1)$ insertion/removal.

22. Implement Rate Limiter

A simple **fixed-window** rate limiter:

</> Java



```
class RateLimiter {

    private final int maxRequests;
    private final long windowMillis;

    private final Map<String, Window> clients = new HashMap<>();

    public RateLimiter(int maxRequests, long windowMillis) {
        this.maxRequests = maxRequests;
        this.windowMillis = windowMillis;
    }

    public synchronized boolean allow(String clientId) {

        long now = System.currentTimeMillis();

        Window window = clients.get(clientId);

        if (window == null ||
            now - window.start >= windowMillis) {

            clients.put(
                clientId,
                new Window(now, 1)
            );

            return true;
        }

        if (window.count < maxRequests) {
            window.count++;
        }
    }
}
```

```

        return true;
    }

    return false;
}

static class Window {
    long start;
    int count;

    Window(long start, int count) {
        this.start = start;
        this.count = count;
    }
}
}

```

Example:

Maximum 100 requests
per 1 minute
per client



Production-level answer

For distributed systems, don't rely on an in-memory HashMap .

Use something like:

API Gateway
↓
Redis
↓
Token Bucket / Sliding Window



Because multiple application instances need to share rate-limit state.

23. Producer-Consumer

Classic Java concurrency question.

Use:

</> Java



BlockingQueue

rather than manually implementing `wait()` / `notify()` .

</> Java



```
class Producer implements Runnable {

    private final BlockingQueue<Integer> queue;

    Producer(BlockingQueue<Integer> queue) {
        this.queue = queue;
    }

    @Override
    public void run() {

        try {
            for (int i = 1; i <= 10; i++) {
                queue.put(i);
                System.out.println("Produced: " + i);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

Consumer:

</> Java



```
class Consumer implements Runnable {

    private final BlockingQueue<Integer> queue;

    Consumer(BlockingQueue<Integer> queue) {
        this.queue = queue;
    }

    @Override
    public void run() {

        try {
            while (true) {
```

```

        Integer value = queue.take();

        System.out.println("Consumed: " + value);
    }
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
}
}

```

Create queue:

</> Java



```

BlockingQueue<Integer> queue =
    new ArrayBlockingQueue<>(10);

```

Why BlockingQueue?

It handles:

Thread safety
Blocking
Waiting
Signalling



without manually writing synchronization logic.

24. Thread-Safe Singleton

Best modern approach: enum

</> Java



```

public enum Singleton {

    INSTANCE;

    public void doSomething() {
        System.out.println("Working...");
    }
}

```

Usage:

</> Java



```
Singleton.INSTANCE.doSomething();
```

Interview question

Why enum Singleton?

Because Java guarantees enum instances are created only once and provides strong serialization guarantees.

Double-Checked Locking

If interviewer specifically asks for class-based Singleton:

</> Java



```
class Singleton {  
  
    private static volatile Singleton instance;  
  
    private Singleton() {  
    }  
  
    public static Singleton getInstance() {  
  
        if (instance == null) {  
  
            synchronized (Singleton.class) {  
  
                if (instance == null) {  
                    instance = new Singleton();  
                }  
            }  
        }  
  
        return instance;  
    }  
}
```

Why volatile?

Because object creation isn't conceptually one indivisible operation. `volatile` prevents unsafe publication/reordering issues here.

25. Custom Immutable Class

Rules:

1. class final
2. fields private final
3. no setters
4. initialize through constructor
5. defensive copy for mutable fields
6. return defensive copies



Example:

`</>` **Java**



```
final class Employee {

    private final int id;
    private final String name;
    private final Date joiningDate;

    public Employee(int id, String name, Date joiningDate) {
        this.id = id;
        this.name = name;
        this.joiningDate =
            new Date(joiningDate.getTime());
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public Date getJoiningDate() {
        return new Date(joiningDate.getTime());
    }
}
```

Why defensive copy?

Because `Date` is mutable.

Without defensive copy:

```
</> Java

Date date = employee.getJoiningDate();

date.setTime(...);
```

could change the internal state of the object.

Modern Java

If appropriate, prefer immutable types such as:

```
</> Java

LocalDate
Instant
LocalDateTime
```

instead of `Date`.

26. Custom Thread Pool

At a high level, a thread pool needs:

```
Task Queue
  ↓
Worker Threads
  ↓
Execute tasks
```

Simple implementation:

```
</> Java

class SimpleThreadPool {

    private final BlockingQueue<Runnable> queue;
    private final List<Thread> workers;
```

```

public SimpleThreadPool(int threadCount) {

    queue = new LinkedBlockingQueue<>();
    workers = new ArrayList<>();

    for (int i = 0; i < threadCount; i++) {

        Thread worker = new Thread(() -> {

            while (!Thread.currentThread().isInterrupted()) {

                try {
                    Runnable task = queue.take();
                    task.run();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
        });

        workers.add(worker);
        worker.start();
    }

    public void submit(Runnable task) {
        queue.offer(task);
    }

}

```

Usage:

</> **Java**



```

SimpleThreadPool pool =
    new SimpleThreadPool(3);

pool.submit(() -> System.out.println("Task 1"));
pool.submit(() -> System.out.println("Task 2"));
pool.submit(() -> System.out.println("Task 3"));

```

But in production?

Don't reinvent it.

Use:

</> Java



```
ExecutorService
```

</> Java



```
ExecutorService executor =  
    Executors.newFixedThreadPool(3);
```

27. Implement Retry Mechanism

Simple retry:

</> Java



```
public static void retry(  
    Runnable task,  
    int maxAttempts,  
    long delayMillis) {  
  
    for (int attempt = 1; attempt <= maxAttempts; attempt++) {  
  
        try {  
  
            task.run();  
  
            return;  
  
        } catch (Exception e) {  
  
            if (attempt == maxAttempts) {  
                throw e;  
            }  
  
            try {  
                Thread.sleep(delayMillis);  
            } catch (InterruptedException ex) {  
                Thread.currentThread().interrupt();  
                throw new RuntimeException(ex);  
            }  
        }  
    }  
}
```

```
}
}
```

Usage:

```
</> Java

retry(
    () -> callExternalService(),
    3,
    1000
);
```

Production-level retry

Don't always use fixed delay.

Use:

```
Exponential Backoff
+
Jitter
+
Maximum Retry Limit
```

Example:

```
1 sec
2 sec
4 sec
8 sec
```

Jitter prevents many services from retrying at exactly the same time.

28. Circuit Breaker Conceptually

Circuit breaker protects your service when a downstream service is failing.

States:

```

      failures
  CLOSED -----> OPEN
      ↑           |

```

```

|               |
| success       | timeout
|               |
|               ↓
+----- HALF_OPEN

```

Simple conceptual implementation

</> Java



```

class CircuitBreaker {

    enum State {
        CLOSED,
        OPEN,
        HALF_OPEN
    }

    private State state = State.CLOSED;

    private int failures = 0;

    private final int failureThreshold = 3;

    public synchronized boolean allowRequest() {

        return state != State.OPEN;
    }

    public synchronized void recordSuccess() {

        failures = 0;
        state = State.CLOSED;
    }

    public synchronized void recordFailure() {

        failures++;

        if (failures >= failureThreshold) {
            state = State.OPEN;
        }
    }
}

```

Real implementation

Usually use:

Resilience4j



with:

Circuit Breaker

Retry

Rate Limiter

Bulkhead

Time Limiter



Interview explanation

Say:

"Circuit breaker prevents repeatedly calling an unhealthy downstream service. After failures cross a threshold, the circuit opens and requests fail fast. After a cooldown period, it enters half-open and allows limited requests to test recovery."

That's a **good 5-year answer**.

29. Top K Frequent Elements

Input:

[1,1,1,2,2,3]



k = 2

Output:

[1,2]



HashMap + PriorityQueue

</> Java



```
public static int[] topKFrequent(int[] nums, int k) {  
  
    Map<Integer, Integer> frequency = new HashMap<>();  
  
    for (int num : nums) {
```

```
        frequency.put(
            num,
            frequency.getDefault(num, 0) + 1
        );
    }

    PriorityQueue<Integer> heap =
        new PriorityQueue<>(
            Comparator.comparingInt(frequency::get)
        );

    for (int num : frequency.keySet()) {

        heap.offer(num);

        if (heap.size() > k) {
            heap.poll();
        }
    }

    int[] result = new int[k];

    for (int i = k - 1; i >= 0; i--) {
        result[i] = heap.poll();
    }

    return result;
}
```

Complexity

Frequency map:

$O(n)$



Heap:

$O(m \log k)$



where m = number of unique elements.

Overall:

$O(n \log k)$



Interview keywords

HashMap + Min Heap

30. Design a Task Scheduler

This is more of a **design** + **coding** question.

A simple scheduler can use:

</> Java



PriorityQueue

ordered by execution time.

</> Java



```
class Task {  
  
    Runnable task;  
    long executeAt;  
  
    Task(Runnable task, long executeAt) {  
        this.task = task;  
        this.executeAt = executeAt;  
    }  
}
```

Scheduler:

</> Java



```
class TaskScheduler {  
  
    private final PriorityQueue<Task> queue =  
        new PriorityQueue<>(  
            Comparator.comparingLong(t -> t.executeAt)  
        );  
  
    public synchronized void schedule(  
        Runnable task,  
        long delayMillis) {
```

```
        queue.offer(
            new Task(
                task,
                System.currentTimeMillis()
                    + delayMillis
            )
        );

        notify();
    }

    public void start() {

        Thread worker = new Thread(() -> {

            while (!Thread.currentThread().isInterrupted()) {

                try {

                    Task task;

                    synchronized (this) {

                        while (queue.isEmpty()) {
                            wait();
                        }

                        task = queue.peek();

                        long delay =
                            task.executeAt
                                - System.currentTimeMillis();

                        if (delay > 0) {
                            wait(delay);
                            continue;
                        }

                        queue.poll();
                    }

                    task.task.run();

                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
        });
    }
}
```

```

        worker.start();
    }
}

```

Usage:

</> Java



```

TaskScheduler scheduler = new TaskScheduler();

scheduler.start();

scheduler.schedule(
    () -> System.out.println("Hello"),
    5000
);

```

The Interview Cheat Sheet

For these questions, memorize the **pattern**, not just the code.

Problem	Best Pattern	Time
Reverse String	Two Pointer / StringBuilder	O(n)
Reverse Words	Split + Reverse	O(n)
Palindrome	Two Pointer	O(n)
Duplicate Characters	HashMap	O(n)
First Non-Repeating	LinkedHashMap	O(n)
Anagram	Frequency Array/Map	O(n)
Character Frequency	HashMap	O(n)
Missing Number	XOR	O(n)
Duplicate Number	Floyd Cycle	O(n)
Move Zeros	Two Pointer	O(n)

Problem	Best Pattern	Time
Second Largest	One Pass	$O(n)$
Merge Arrays	Two Pointer	$O(n+m)$
Intersection	HashSet	$O(n+m)$
Rotate Array	Reversal	$O(n)$
Maximum Subarray	Kadane	$O(n)$
Longest Unique Substring	Sliding Window	$O(n)$
Product Except Self	Prefix + Suffix	$O(n)$
Three Sum	Sort + Two Pointer	$O(n^2)$
Merge Intervals	Sort + Merge	$O(n \log n)$
Valid Parentheses	Stack	$O(n)$
LRU Cache	HashMap + DLL	$O(1)$
Rate Limiter	Token Bucket/Sliding Window	$\sim O(1)$
Producer Consumer	BlockingQueue	$O(1)$ per operation
Singleton	Enum / DCL	$O(1)$
Immutable Class	Final + Defensive Copy	—
Thread Pool	Queue + Workers	—
Retry	Backoff + Retry	—
Circuit Breaker	State Machine	—
Top K Frequent	HashMap + Min Heap	$O(n \log k)$
Task Scheduler	PriorityQueue + Worker	$O(\log n)$



Patterns You Should Memorize for a 5-Year Interview

Instead of remembering 30 completely different solutions, remember these 10 patterns:

1. Two Pointer

Used in:

Palindrome
Move zeros
Two Sum on sorted array
Three Sum
Merge sorted arrays



2. Sliding Window

Used in:

Longest substring
Maximum/minimum window
Subarray problems



3. HashMap / HashSet

Used in:

Frequency
Duplicates
Anagrams
Intersection
Two Sum



4. Stack

Used in:

Valid parentheses
Next greater element



Expression problems

5. Binary Search

Used when:

Array is sorted
OR
answer space is searchable



6. Heap / PriorityQueue

Used in:

Top K
Kth largest
Task scheduler
Merge K sorted lists



7. Prefix / Suffix

Used in:

Product except self
Range queries
Prefix sum



8. Fast & Slow Pointer

Used in:

Linked-list cycle
Duplicate number
Middle of linked list



9. Sort + Greedy

Used in:

Merge intervals
Three sum
Meeting rooms
Activity selection



10. Concurrency Building Blocks

For your **5-year Java interview**, be especially comfortable with:

BlockingQueue
ExecutorService
CompletableFuture
ConcurrentHashMap
AtomicInteger
synchronized
Lock
Semaphore
CountDownLatch
CyclicBarrier
volatile



The most important upgrade from a junior answer to a **5-year-level answer** is:

Don't stop at "this code works." Explain why it is optimal, what happens under concurrency, what happens at scale, and what you would use in production.

For example, for **LRU Cache**, don't just say `LinkedHashMap` . Say:

"For a simple in-process implementation I can use access-ordered `LinkedHashMap`. If I need to implement it from scratch, I would combine `HashMap` for $O(1)$ lookup with a doubly linked list for $O(1)$ eviction. In a distributed environment, I would consider a distributed cache rather than keeping the state inside one JVM."

That distinction is exactly the kind of answer interviewers look for at **4–5 years of experience**.



ChatGPT can make mistakes. Check important info.